

Anomaly Detection in Time Series – Multivariate Analysis & Autoencoders



Principal Component Analysis for Anomaly Detection

Most anomaly detection methods operate on a single time series. When multiple correlated variables are observed simultaneously, anomalies may only be visible in the joint behavior — not in any individual series.

How PCA works for anomaly detection:

- 1. PCA learns a low-dimensional subspace that captures the normal variance of the data
- 2. Each new observation is projected onto this subspace and reconstructed
- 3. The reconstruction error (residual) measures how far the observation deviates from the normal subspace
- 4. Observations with high reconstruction error are flagged as anomalies

Input	Model	Anomaly score
Multivariate dataframe (multiple simultaneous series)	Linear projection onto principal components	Reconstruction error (L2 norm of residual)
Threshold		Harbinger function
Automatically estimated from training residuals		<code>hmu_pca()</code>

❏ Ref: Jolliffe, I.T. (2002). Principal Component Analysis. Springer, 2nd ed. | Shyu et al. (2003). A Novel Anomaly Detection Scheme Based on Principal Component Classifier. ICDM.

41 · hmu_pca() – Multivariate Detection via PCA

```
library(daltoolbox)
library(harbinger)
library(ggplot2)

# Carregar dataset multidimensional
data("examples_harbinger")
dataset <- examples_harbinger$multidimensional

# Criar labels de evento (para demo)
dataset$event <- FALSE
dataset$event[c(101, 128, 167)] <- TRUE

head(dataset)
## serie      x      event
## 1 -0.6264538 0.40940184 FALSE
## 2 -0.8356286 1.58658843 FALSE

# Visualizar série principal
har_plot(harbinger(), dataset$serie)

# Visualizar segunda dimensão
har_plot(harbinger(), dataset$x)

# Ajustar PCA nas 2 primeiras colunas
model <- fit(hmu_pca(),
             dataset[, 1:2])
```

```
# Detectar anomalias
detection <- detect(model,
                   dataset[, 1:2])

# Plotar detecções na série principal
grf <- har_plot(model,
               dataset$serie, detection,
               dataset$event)
grf <- grf + ylab("serie")

# Plotar detecções na 2ª dimensão
grf <- har_plot(model,
               dataset$x, detection,
               dataset$event)
grf <- grf + ylab("x")

# Plotar magnitude residual
har_plot(model,
         attr(detection, "res"),
         detection, dataset$event,
         yline = attr(detection,
                     "threshold"))
```

Step-by-step

- Uses `examples_harbinger$multidimensional` (2 variables)
- Event labels created manually for the demo: events at `idx=101, 128, 167`
- Visualize each dimension separately before modeling
- `hmu_pca()`: projects observations onto principal components
- `fit()` and `detect()` receive a dataframe with multiple columns
- PCA reconstruction error serves as the anomaly score
- Plot detections for each dimension separately using `ylab()`

📌 **Key concept:** PCA projects multivariate data onto principal components and reconstructs it. Observations with a large reconstruction error — those far from the learned normal subspace — are flagged as anomalies. This extends detection to multiple simultaneous variables. Ref: Jolliffe, 2002.

Autoencoders – Reconstruction-Based Anomaly Detection

An autoencoder is a neural network trained to compress an input into a compact latent representation (encoding) and then reconstruct it (decoding). When trained exclusively on normal data, the network learns to reconstruct normal patterns well — but fails on anomalous inputs, producing a high reconstruction error.

The anomaly detection pipeline:

1. Train the autoencoder on normal (non-anomalous) windows of the time series
2. For each new window, compute the reconstruction error
3. If the error exceeds a learned threshold, flag the window as anomalous

Note: In Harbinger, all autoencoder variants are accessed via `han_autoencoder(encoder_size, decoder_size, type, num_epochs)` — only the `type` argument changes.

autoenc_ed

Basic: dense encoder-decoder baseline

autoenc_denoise_ed

Denoising: trained on corrupted inputs for noise robustness

autoenc_conv_ed

Convolutional: Conv1D layers for local pattern detection

autoenc_lstm_ed

LSTM: recurrent layers for long-range temporal dependencies

autoenc_stacked_ed

Stacked: multiple hidden layers for deep representations

autoenc_variational_ed

VAE: probabilistic latent space (μ , σ^2)

autoenc_adv_ed

AAE: adversarial regularization of the latent space

34 · Basic Autoencoder – Window Reconstruction

```
library(daltoolbox)
library(daltoolboxdp)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Basic Autoencoder
# args: encoder_size, decoder_size,
#      type, num_epochs
model <- han_autoencoder(
  3, 2, autoenc_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Result: idx=51 (0 TP, 1 FP)
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##    TRUE  0   1
##   FALSE  1  99

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by step – Basic Autoencoder

- Requires `daltoolboxdp` in addition to `daltoolbox`
- Arguments: encoder size (3), decoder size (2), type, number of epochs
- Reconstructs sliding windows; high reconstruction error signals an anomaly
- Detected idx=51 (close to ground truth at 50): 0 TP, 1 FP
- A one-step offset is common with basic autoencoders

❏ **Key concept:** Autoencoders reconstruct windows; a high reconstruction error indicates an anomaly. The encoder compresses the window into a latent representation; the decoder attempts to reconstruct it. Ref: Sakurada & Yairi, 2014.

Denoising Autoencoder – Learning Robust Representations

A Denoising Autoencoder (DAE) is trained to reconstruct clean inputs from artificially corrupted versions. By learning to ignore noise during training, the model develops a more robust internal representation of normal patterns.

How it differs from the basic autoencoder:

- Basic AE: input = clean window → reconstruct clean window
- Denoising AE: input = corrupted window → reconstruct clean window

Training procedure:

1. For each training window, add random noise (e.g., Gaussian or masking noise)
2. Feed the corrupted window to the encoder
3. Train the decoder to reconstruct the original clean window
4. At inference time, feed the original (uncorrupted) window — high reconstruction error signals an anomaly

Key advantage: more robust to measurement noise and sensor jitter in real-world time series.

When to prefer DAE

Series with significant measurement noise, sensor drift, or environmental interference where a basic AE would produce too many false positives.

Limitation

On clean synthetic datasets, DAE and basic AE may produce identical results — the advantage only emerges with real-world noisy data.

❏ Ref: Vincent et al. (2010). Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion. JMLR, 11, 3371–3408.

35 · Denoising Autoencoder – Noise Robustness

```
library(daltoolbox)
library(daltoolboxdp)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Autoencoder Denoising
model <- han_autoencoder(
  3, 2, autoenc_denoise_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Resultado: idx=51 (0 TP, 1 FP)

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99
```

```
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by step – Denoising AE

- Learns to reconstruct clean inputs from noisy windows
- Trained with artificially corrupted inputs
- More robust to noise than the basic autoencoder
- Same result on this dataset: idx=51, 0 TP, 1 FP
- The difference emerges on series with real-world noise

 **Key concept:** A denoising autoencoder adds robustness by training on corrupted inputs — it learns to ignore ordinary noise and focus on the underlying structure. This provides a real advantage on series affected by measurement noise.

Convolutional Autoencoder – Local Pattern Detection

A Convolutional Autoencoder (Conv AE) replaces the dense layers of a standard autoencoder with 1D convolutional filters (Conv1D). This architecture is specifically designed to capture local patterns and short-range dependencies within a time window.

How Conv1D works in this context:

- Filters slide over the time window, detecting local features (peaks, drops, shape changes)
- Translation invariance: the same feature is detected regardless of its exact position within the window
- Encoder: Conv1D layers progressively compress the window into a latent representation
- Decoder: transposed Conv1D layers reconstruct the window from the latent code

Anomaly score: reconstruction error — windows with unusual local shapes produce high error.

Best suited for: point anomalies and shape anomalies with a localized signature.

Advantage over Basic AE

Captures local temporal structure; translation-invariant detection

Advantage over LSTM AE

More precise temporal localization of the anomaly index

Best for

Anomalies with a distinct local shape (spikes, drops, pulses)

Result on simple dataset

1 TP, 0 FP — best result among all autoencoder variants

📄 Ref: Baldi, P. (2012). Autoencoders, Unsupervised Learning, and Deep Architectures. ICML Workshop on Unsupervised and Transfer Learning.

36 · Convolutional Autoencoder – Local Patterns

```
library(daltoolbox)
library(daltoolboxdp)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Autoencoder Convolutional
model <- han_autoencoder(
  3, 2, autoenc_conv_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – Conv Autoencoder

- Uses Conv1D layers in both the encoder and decoder
- Captures local patterns and provides translation invariance
- Perfect result: 1 TP, 0 FP (detected exactly idx=50)
- Outperforms the basic autoencoder on this dataset
- Best suited for anomalies with a localized shape

📌 **Key concept:** The convolutional autoencoder uses Conv1D filters to capture local patterns within the time window. Translation invariance makes it robust to small temporal shifts in the position of an anomaly.

LSTM Autoencoder – Capturing Temporal Dependencies

An LSTM Autoencoder replaces the dense or convolutional layers with Long Short-Term Memory (LSTM) units. LSTMs are recurrent networks designed to model sequential data by maintaining a hidden state that carries information across time steps.

How it works:

- Encoder LSTM: reads the input window step by step, compressing the entire sequence into a fixed-size context vector (latent state)
- Decoder LSTM: reconstructs the sequence from the context vector, step by step
- The model is trained to minimize reconstruction error on normal sequences
- At inference, sequences that deviate from learned temporal patterns produce high reconstruction error

Key property: the latent representation encodes the temporal order and long-range context of the entire window — not just local features.

Best suited for: contextual anomalies where the anomaly is defined by its relationship to surrounding values over a long horizon.

Strength

Long-range temporal context; sensitive to changes in sequence dynamics

Limitation

May exhibit a one-step temporal offset in the detected anomaly index

Contrast with Conv AE

LSTM captures global sequence context; Conv AE captures local shape

Best for

Contextual anomalies, regime changes, slow drifts

📄 Ref: Malhotra et al. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. ICML Anomaly Detection Workshop.

37 · LSTM Autoencoder – Temporal Dependencies

```
library(daltoolbox)
library(daltoolboxdp)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Autoencoder LSTM
model <- han_autoencoder(
  3, 2, autoenc_lstm_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Resultado: idx=51 (0 TP, 1 FP)
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   1
##   FALSE  1  99

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – LSTM Autoencoder

- LSTM learns temporal dependencies by encoding and decoding full sequences
- Optimizes reconstruction of complete time sequences
- Detected idx=51 (0 TP, 1 FP) — one-step offset from the true anomaly
- Contrast with Conv AE: LSTM was less precise on this dataset
- Best suited for long-range contextual anomalies

📌 **Key concept:** The LSTM autoencoder captures long-range temporal dependencies in the latent representation. It excels at contextual anomalies — but may exhibit a temporal shift in the detection index.

Stacked Autoencoder – Deep Hierarchical Representations

A Stacked Autoencoder extends the basic autoencoder by adding multiple hidden layers in both the encoder and decoder. Each additional layer allows the network to learn increasingly abstract representations of the input data.

Architecture:

- Encoder: input → hidden layer 1 → hidden layer 2 → ... → latent code (bottleneck)
- Decoder: latent code → hidden layer n → ... → reconstructed output
- Each layer learns a progressively more abstract feature of the normal pattern

Why depth matters for anomaly detection:

- Shallow autoencoders may miss subtle, non-linear anomalies
- Deeper architectures capture complex structure that simple models cannot represent
- The richer latent space makes the model more sensitive to deviations from normal behavior

Trade-off: higher sensitivity comes at the cost of more false positives — threshold tuning is essential.

Depth advantage

Captures non-linear, hierarchical structure in normal patterns

Sensitivity

Detects subtle anomalies that shallow models miss

Risk

Higher false positive rate — requires careful threshold calibration

Result on simple dataset

1 TP, 2 FP (idx=26, 50, 77 detected)

📖 Ref: Bengio, Y. et al. (2007). Greedy Layer-Wise Training of Deep Networks. NeurIPS. | Vincent et al. (2010). Stacked Denoising Autoencoders. JMLR.

38 · Stacked Autoencoder – Deep Representation

```
library(daltoolbox)
library(harbinger)
library(daltoolboxdp)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Stacked Autoencoder
model <- han_autoencoder(
  3, 2, autoenc_stacked_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
```

```
print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 26 TRUE anomaly
## 2 50 TRUE anomaly
## 3 77 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE 1 2
##   FALSE 0 98

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step – Stacked AE

- Deepens encoder/decoder layers to capture richer non-linear structure
- Detected 3 events (idx=26, 50, 77): 1 TP, 2 FP
- Deeper architecture → higher sensitivity, but also more false positives
- Threshold tuning required to reduce FP rate
- Well-suited for detecting subtle anomalies

 **Key concept:** Stacked AE deepens the latent representation through multiple layers, capturing richer non-linear structure. This makes it more sensitive to subtle anomalies, but threshold tuning is essential to keep false positives in check.

Variational Autoencoder (VAE) – Probabilistic Latent Space

A Variational Autoencoder (VAE) is a generative model that extends the standard autoencoder by learning a probabilistic latent space instead of a fixed point encoding. Rather than mapping each input to a single latent vector, the encoder outputs a distribution — parameterized by a mean (μ) and variance (σ^2).

How it works:

- Encoder outputs μ and σ^2 for each input window
- A latent vector z is sampled from $N(\mu, \sigma^2)$ using the reparameterization trick
- Decoder reconstructs the window from z
- Loss = reconstruction error + KL divergence (regularizes the latent distribution toward $N(0,1)$)

Anomaly detection: anomalous windows produce both high reconstruction error AND high uncertainty (large σ^2) — two complementary signals.

Key advantage over deterministic AEs: the VAE quantifies uncertainty in its detections, which can be used to rank or filter alerts.

Latent space

Continuous, smooth, probabilistic — enables interpolation between normal patterns

Anomaly signals

Reconstruction error + KL divergence / uncertainty

Advantage

Uncertainty quantification — anomalies are flagged with a confidence measure

Result on simple dataset

1 TP, 2 FP (idx=50, 58, 85)

39 · Variational Autoencoder (VAE) – Probabilistic Latent Space

```
library(daltoolbox)
library(harbinger)
library(daltoolboxdp)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# VAE - Variational Autoencoder
model <- han_autoencoder(
  3, 2, autoenc_variational_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
## 2 58 TRUE anomaly
## 3 85 TRUE anomaly

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1  2
##   FALSE 0 98

har_plot(model, dataset$serie,
  detection, dataset$event)
```

Step-by-step – VAE

- Learns a probabilistic latent space (mean + variance)
- Reconstruction error and uncertainty signal anomalies
- Detected 3 events (idx=50, 58, 85): 1 TP, 2 FP
- Similar sensitivity to Stacked AE
- Advantage: quantifies uncertainty in detection

📌 **Key concept:** The VAE learns a probabilistic latent distribution (μ , σ^2). Anomalies exhibit high uncertainty and elevated reconstruction error. Ref: Kingma & Welling, 2013.

Autoencoders – Architecture Comparison

Basic (autoenc_ed)

Detection: idx=51. 0 TP, 1 FP.
Baseline reference.

Denoising (autoenc_denoise_ed)

Detection: idx=51. 0 TP, 1 FP.
Robust to noise.

Convolutional (autoenc_conv_ed)

Detection: idx=50. 1 TP, 0 FP.
Best result ✓

LSTM (autoenc_lstm_ed)

Detection: idx=51. 0 TP, 1 FP.
Suited for long-range dependencies.

Stacked (autoenc_stacked_ed)

Detection: idx=26, 50, 77. 1 TP, 2 FP. Higher sensitivity.

Variational (autoenc_variational_ed)

Detection: idx=50, 58, 85. 1 TP, 2 FP. Probabilistic latent space.

📌 **Key Takeaway:** The Convolutional Autoencoder achieved the best result on this dataset. Deeper architectures tend to be more sensitive, but also more prone to false positives — threshold tuning is essential.

Adversarial Autoencoder (AAE) – Adversarial Latent Regularization

An Adversarial Autoencoder (AAE) combines an autoencoder with a Generative Adversarial Network (GAN)-style discriminator applied to the latent space. Instead of using KL divergence (as in VAE) to regularize the latent distribution, the AAE uses adversarial training to force the encoder's output to match a chosen prior distribution (e.g., Gaussian).

Architecture — three components:

1. **Encoder:** maps input windows to latent vectors z
2. **Decoder:** reconstructs the window from z (minimizes reconstruction error)
3. **Discriminator:** distinguishes between latent vectors z produced by the encoder and samples drawn from the prior — the encoder is trained to fool the discriminator

Anomaly detection: normal windows map to the learned manifold and reconstruct well. Anomalous windows fall outside the manifold, producing high reconstruction error.

Key difference from VAE: adversarial regularization is more flexible and can match any prior distribution, not just Gaussian.

Regularization

Adversarial (GAN-style) instead of KL divergence

Prior

Any distribution (Gaussian, mixture, etc.) — more flexible than VAE

Anomaly signal

Reconstruction error from the decoder

Limitation

Training instability common to GAN-based models; sensitive to hyperparameters

Result on simple dataset

0 TP, 2 FP (idx=19, 44)

📄 Ref: Makhzani, A. et al. (2016). Adversarial Autoencoders. arXiv:1511.05644.

40 · Adversarial Autoencoder (AAE)

```
library(daltoolbox)
library(daltoolboxdp)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$serie

har_plot(harbinger(), dataset$serie)

# Definir AAE
# Encoder + Decoder + Discriminator
model <- har_autoencoder(
  3, 2, autoenc_adv_ed,
  num_epochs = 1500)

model <- fit(model, dataset$serie)
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 19 TRUE anomaly
## 2 44 TRUE anomaly

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 0 2
## FALSE 1 98

har_plot(model, dataset$serie,
  detection, dataset$event)

har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step

- AAE augments the encoder-decoder with an adversarial objective applied to the latent code
- A discriminator distinguishes latent samples produced by the encoder from samples drawn from a prior distribution (e.g., Gaussian)
- The encoder is trained to fool the discriminator — aligning the posterior with the prior
- Normal patterns remain on a compact manifold; anomalies exhibit higher reconstruction error
- Detected idx=19 and 44 (0 TP, 2 FP on this dataset)

📌 **Key concept:** AAE regularizes the latent space through adversarial training, forcing the latent distribution to follow a prior. Windows that deviate from the learned manifold yield elevated reconstruction error. Ref: Makhzani et al., arXiv 2016.

Summary: Method Families and When to Use Them

Fundamentals

`hanr_histogram()`

Use as your **first baseline**. No model parameters — focuses on the empirical distribution. Fast and interpretable.

Supervised

`hanr_ml()` + `hanc_ml()`

Use when **labels are available**. Regression: prediction errors. Classification: positive class probability.

Ensembles

`har_ensemble()`

Use after becoming familiar with simpler families. **Reduces false positives** by combining multiple detectors.

Multivariate

`hmu_pca()`

Use when anomalous behavior emerges from the **relationship between variables**, rather than from a single isolated series. PCA reconstruction error serves as the anomaly score.

Residual Models

ARIMA, FBIAD, EMD, FFT*, GARCH, REMD, RTAD, Wavelet

Use when there is a **known temporal structure**. Anomalies are deviations from the behavior expected by the model.

Clustering

k-means, DTW

Use when **similarity between subsequences** matters more than a predictive model. Detects point anomalies and discords.

Autoencoders (Deep Learning)

Basic, Denoising, Convolutional, LSTM, Stacked, VAE, AAE — all via `han_autoencoder(type=...)`

Use to capture **complex non-linear structure**. Conv AE excels at local patterns; LSTM at long-range dependencies; VAE adds probabilistic uncertainty; AAE regularizes via adversarial training.

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Sakurada, M. & Yairi, T. (2014). Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction. *MLSDA Workshop*.

Kingma, D.P. & Welling, M. (2013). Auto-Encoding Variational Bayes. *arXiv:1312.6114*.

Makhzani, A. et al. (2016). Adversarial Autoencoders. *arXiv:1511.05644*.

Jolliffe, I.T. (2002). *Principal Component Analysis*. Springer, 2nd ed.